How to get SteamVR working on Linux

including standard UEVR games, Satisfactory with Dortamur's mod, and Resident Evil Games with REFramework

--For Quest 3 + Linux Mint 22.3 + Nvidia 5080--

--Wireless AND Cabled--

by FutureOnVR

v1.1 - First viable Public release version (Finalised 16th April 2026)

Contents:

- ✔ Acknowledgements: **Page 2**.
- ✔ Linux Mint installation + setup: **Pages 3-7**.
- ✔ ALVR setup: **Pages 7-9**.
- ✔ How to play SteamVR native games on Quest 3: **Pages 9-10**.
- ✔ **Reference 1.0**: How to connect to SteamVR over **Wireless ALVR** each time: **Page 10**.
- ✔ ALVR Cabled Setup and configuration: **Pages 11-15**.
- ✔ **Reference 1.1**: How to connect Quest 3 to SteamVR (via cable) each time: **Page 13**.
- ✔ **Reference 1.2**: Best-quality ALVR+SteamVR settings: **Pages 14-15**.
- ✔ How to Add and Pin a Window in-game using Quest 3, ALVR, & WayVR: **Pages 17-19**.

PART 2: UEVR-BASED MODDING GUIDE

- ✔ UEVR first step and important note: **Page 20.**
- ✔ **UEVR** Game Tutorial 1**: Satisfactory** + Dortamur's mod**: Pages 20-26**.
- ✔ **Reference 2.1: Satisfactory startup steps**, each time: **Page 26**.
- ✔ **UEVR** Game Tutorial 2**: GTA San Andreas** Definitive Edition: **Pages 27-31**.
- ✔ How to try different Proton versions (inc. GE Proton) with UEVR games: **Pages 28-31**.
- ✔ **UEVR** Game Tutorial 3**: Resident Evil 2** (remake): **Pages 32-34**.

Acknowledgements

My sincere appreciation to:

- **DipiVR**, for **showing me it was possible** to play VR games on Linux, for his help in suggesting **first steps** to get it working on Quest 3, as well as **providing the overall steps** required to get **Satisfactory VR** working, which I adapted to Quest 3 to 5080 via ALVR, and for **tips, pointers and solutions** to getting some of the things in this guide to work!

- **Dortamur**, not only for **his awesome Satisfactory VR mod**, but for **encouraging me** with the potential usefulness of a guide to those interested in trying VR on Linux or even streaming with it.

- **LiveOpenMike**, for being **a general inspirer for all things VR**, and especially for motivating me to keep playing VR when my interest was waning.

- **KroyVR**, for similar inspiration, but much more **PCVR focused**, and mighty discoverer of great PCVR games!

  You can find them all on Twitch.

Linux Mint Installation, Setup, and Configuration

1. Prepare the install USB

Note: If you want to keep Windows, either move it to a slower drive, or have another NVMe drive available to install Linux to.

Download the Linux Mint Cinnamon Edition ISO from https://www.linuxmint.com/download.php

Verify its sha256 hash on Windows using the Windows Store app Hash Checker by Matt Cooley, before installing.

Download Balena Etcher from https://etcher.balena.io/

Plug in your USB stick of at least 8GB in capacity.

Run Balena Etcher, choose the Linux Mint iso file and choose the USB stick. Flash.

Once finished, restart your PC, repeatedly tap your motherboard's BIOS activation key (for me it's the Delete key) until you're in BIOS settings - set the PC to boot from the still plugged in USB stick before Windows. F10 to save and exit.

Then boot to the USB and install Mint.

2. During Mint installation

For your Linux install drive, I'd use:

- **ext4**
- **full-disk encryption enabled** - does not slow down gaming.
- **do not also encrypt only the home folder** - that would slow things down.

Mint's install guide includes the encryption options during installation.

If you have additional game SSDs:

- **btrfs is fine, or stick to ext4.**
- it does **not** need to match the OS drive filesystem
- the important thing is that they are a normal Linux filesystem, not NTFS, for your active Linux Steam libraries.

Your games do **not** need to be on the same drive as Mint.

What matters is the **filesystem**, not whether it's the OS disk, and the speed. All games you're going to play in VR must be on NVMe drives. Install flatscreen games to slower SSDs or HDDs.

3. First boot: fully update Mint

Open Terminal.

First, turn on the firewall and make sure it's set to deny all incoming traffic and allow all outgoing traffic:

- sudo ufw enable

Check that the deny-incoming and allow-outgoing rules are on by default with:

- sudo ufw status verbose

There it will confirm if active, and the Defaults line will show the deny/allow rules.

Second, fully update Mint:

- sudo apt update
- sudo apt full-upgrade -y
- sudo reboot

Do this before touching VR.

4. Install the NVIDIA proprietary driver

For your setup, **this is necessary** for serious VR use. Valve says SteamVR on Linux needs up-to-date graphics drivers, and ALVR's Linux setup depends on the proper GPU driver path for hardware encoding. Valve specifically says NVIDIA drivers should be **525+** on Linux.

On Mint, the easiest route is:

GUI route (Recommended)

Open:

Menu → Administration → Driver Manager

Let it scan, then choose the recommended proprietary NVIDIA driver (usually the first entry, even though it may be mislabeled as open-source), apply it, and reboot.

Terminal route

If needed:

- ubuntu-drivers devices
- sudo ubuntu-drivers install
- sudo reboot

Either way, you can check that your Nvidia GPU is being used by Steam and SteamVR by opening a terminal, issuing this command, and leave it running:

- *nvidia-smi dmon*

When a SteamVR game is actually using the NVIDIA GPU, you should usually see:

- ***GPU-Util*** *rise above idle*
- ***Power*** *increase*
- ***Memory usage*** *go up*
- *one or more* ***processes*** *appear using the GPU*
- ***Check the Processes section while the game is running***
  In nvidia-smi, look at the bottom. If SteamVR, the game, or a VR streaming/encoding process appears there while performance and memory rise, that is strong evidence the NVIDIA GPU is doing the work. NVIDIA documents nvidia-smi as the standard tool for monitoring per-GPU utilization.

Also, if you're on a hybrid CPU-GPU (like the AMD 9800X3D) plus a dedicated NVidia GPU, you'll see later where I have added launch parameters to ensure the 5080 is being used. Another option is to simply turn off the CPU's GPU in BIOS entirely, which is the simplest way to ensure this. ***This is not required if your only GPU is the dedicated one****.*

5. Enable 32-bit support for Steam compatibility

Enable i386 support:

- sudo dpkg --add-architecture i386
- sudo apt update

Then install the matching 32-bit NVIDIA GL package for your installed driver branch, as follows:

First check your branch:

- nvidia-smi

Then install the matching package, in my case it's version 580, so:

- sudo apt install libnvidia-gl-580:i386

Use your actual installed version number, not 580 blindly.

6. Install Steam & SteamVR

Use the official Steam package from Mint/Ubuntu repos or Steam's official Linux installer. Then install **SteamVR**, launch it once, and close it. ALVR's installation guide explicitly says to install SteamVR and launch it once before continuing.

Optional: Harden Mint without wrecking usability

- sudo apt install apparmor apparmor-utils unattended-upgrades
- sudo systemctl enable --now apparmor
- sudo dpkg-reconfigure unattended-upgrades

That gives you a meaningfully stronger baseline without turning the setup into a science project.

Note: My guide was written without installing apparmor.

Optional 2: Set Terminal to PASTE with Ctrl+V instead of Shift+Ctrl+V

This will save you a lot of finger-gymnastics pasting in Terminal commands. To set it to Ctrl+v permanently, do this:

Open **Terminal**

1. Click the **menu** button in the top-right of Terminal
2. Open **Preferences**
3. In the sidebar, click **Shortcuts**
4. Make sure **Enable shortcuts** is on
5. Click just to the right of **Paste**
6. Press **Ctrl+v**

This change is saved automatically and stays in place for all future Terminal windows and sessions!

While you're at it, follow the same steps but for Copy - click next to it and press Ctrl and "c".

Just note that the only times it would not persist across shutdown-startup cycles are if:

- you later reset Terminal preferences
- your home/profile settings get wiped or restored from backup
- or Linux Mint swaps you to a different terminal app than the one you changed.

A small gotcha: some terminal apps intercept Ctrl+V differently, and in terminal programs like vim, nano, tmux, or screen, pasting behavior can still depend on that app's own key handling. But the Terminal shortcut itself should remain saved after reboot.

7. Install ALVR on Quest 3 from the in-headset store

8. Enable Developer Mode on the Quest 3

Use the Meta Horizon app on your phone and make sure **Developer Mode** is enabled for the headset. ALVR's installation flow still assumes developer mode for Quest/Pico headset-side setup.

9. Install ALVR on Linux Mint

Install the Linux ALVR streamer on the PC.

ALVR supports Linux, and its installation/troubleshooting docs are the right reference for setup and fixing session issues.

Go to:

https://github.com/alvr-org/ALVR/wiki/Installation-guide

read the instructions for Manually installing ALVR streamer (launcher is just for the Quest 3 version via Linux, which is not needed): Linux.

1. Download “alvr_streamer_linux.tar.gz” from the linked download page.

2. Extract it by right-click: Open with Archive Manager: Click Extract: Choose Home user folder, create new folder called VRonMint and extract to there.

3. Run bin/alvr_dashboard: Right-click empty space in the VRonMint folder and click Open in Terminal. In the terminal copy and paste:

- ./bin/alvr_dashboard

This opens the ALVR Setup Wizard. Click next until you get to the Firewall page. Click Add firewall rules. If successful, a line of feedback appears at the botton saying “Setting firewall rules succeeded!” Click Next.

Keep clicking next until you reach the ALVR Dashboard screen which has a prompt to start SteamVR, and lists things like Wired Connection, New Wireless Devices, and Trusted Wireless Devices, etc.

4. **Make ALVR into a REAL app**, launchable from the Menu, and pin-able to the panel or dock, if using one. First, open the Terminal and enter this command:

- xed ~/.local/share/applications/alvr-dashboard.desktop

Paste in the following, replacing (**YourUsername**) with your actual home folder username.

```
[Desktop Entry]
Version=1.0
Type=Application
Name=ALVR Desktop
Comment=Launch & Play SteamVR on Linux
Exec=/home/(YourUsername)/VRonMint/alvr_streamer_linux/bin/alvr_dashboard
Icon=alvr
Terminal=false
Categories=Utility;
```

------Save and close the text editor. **Note**: Exec must be an absolute path. It cannot be relative.

Now, update the app database, and restart Cinnamon:

- update-desktop-database ~/.local/share/applications

- cinnamon -r &

Now, you can search for ALVR in the Menu, launch it, and then pin it to the panel or dock for ease of access and quick-launch-ability!

10. Network setup for best wireless quality

(skip straight to step 15 if you only want to ever use a cabled connection to your Quest 3, e.g. if your WiFi network is not dedicated or strong).

ALVR recommends:

- the **PC on wired Ethernet**
- the headset on **5 GHz / 802.11ac or better**
- both connected to the **same router**

So your ideal setup is:

- PC → Ethernet to router
- Quest 3 → Wi-Fi 6/6E to same router

That gives you the best chance of matching or beating your old Mint result.

11. How To Play SteamVR-Native Games on Quest 3:

Note: This is using the **Default Public Version of SteamVR** (at time of writing == the March 25 2026 update).

a) **Open Steam**.

Add this to launch options box for SteamVR via steamVR right-click: Properties:

```
__GLX_VENDOR_LIBRARY_NAME=nvidia __NV_PRIME_RENDER_OFFLOAD=1
VK_DRIVER_FILES=/usr/share/vulkan/icd.d/nvidia_icd.json ~/.steam/debian-
installation/steamapps/common/SteamVR/bin/vrmonitor.sh %command%
```

Next, **only IF you have two GPUs**, (one CPU-based, one dedicated), right-click: Properties **on each SteamVR native game** and add this launch option:

```
__NV_PRIME_RENDER_OFFLOAD=1 __VK_LAYER_NV_optimus=NVIDIA_only
__GLX_VENDOR_LIBRARY_NAME=nvidia
VK_DRIVER_FILES=/usr/share/vulkan/icd.d/nvidia_icd.json %command%
```

b) **Launch SteamVR**, then still on PC, open SteamVR settings (three horizontal lines in top right of little SteamVR box), and go to OpenXR: set SteamVR as the Current OpenXR Runtime.

Note, on subsequent launch attempts of SteamVR, if it launches and crashes, just make sure you've Quit out of ALVR on the Quest 3 first, then try again.

First as an initial setup, start SteamVR to establish initial files etc. Quit it. Then...

Reference 1.0: How to connect to SteamVR over Wireless ALVR each time:

- Start **Steam**
- Start **ALVR** on the Quest 3 (no cable between Quset 3 and PC).
- Start **ALVR** on the PC - **ensure the bitrate** in Settings: Video **is reasonable** (e.g. 200-300Mbps) - if it's too high (e.g. 1000Mbps), it simply won't load SteamVR in-headset.
- In ALVR Dashboard on Mint, under New Wireless Devices, click "Trust" next to the headset reference (e.g. "4925.client").
- Wait for it to connect and launch SteamVR automatically (assuming Reference 1.2 step 4 is set). Otherwise, simply click Launch SteamVR manually, after connection.

Note: Set ALVR Dashboard settings to those found in "**Reference 1.2: Best-quality ALVR+SteamVR settings**" below, except, remember to lower the bitrate to 200-300Mbps for Wireless, and increase from there if still stable.

---

12. If ALVR is acting weirdly, consistently, reset its config:

ALVR's troubleshooting page says the first thing to try is deleting session.json in the ALVR installation folder on the PC, then restarting ALVR.

So if pairing, video, or controls act bizarrely:

- close ALVR, SteamVR and Steam, and even restart the PC and relaunch all to see if that fixes it.
- Otherwise, delete "session.json" found in ~/.config/alvr
- Relaunch ALVR Dashboard and re-pair the Quest 3.

13. If wireless graphics quality still looks significantly worse than PCVR with Quest 3 on Windows, use ALVR cabled:

ALVR has a wired USB mode. USB 2.0 works, but 3.0+ is better. Cabled ALVR **supports a higher bandwidth than wireless**, and is said to be the more stable and robust cabled solution compared to an alternative Quest-3-to-Linux-PC solution called WiVRn.

***Note**: WiVRn isn't included in v1.1 of this guide due to the warnings reported with Nvidia GPUs and the time it would take to implement this ALVR replacement. However, I will be including a full tutorial on how to get it working, in a subsequent version of this guide, due to both my curiosity, to see if it actually works on Linux Mint with a 5080, and how well, and because it provides a much more functional WayVR implementation, allowing for things such as resizable windows, useful for streamers, if they do no want to live with standard widescreen window options behind their wrist or pinned somewhere such as above normal view, etc.*

Preparation steps:

Pair your Quest 3 with your Meta Horizon mobile app, if not done so already.

Then, enable developer mode in the app for the Quest 3:

Tap Meta Quest 3 drop-down arrow: Headset Settings: Developer Mode: switch it on.

Now, get "ADB" on Linux Mint to allow functionality:

- sudo apt update
- sudo apt install android-sdk-platform-tools

Then:

- adb kill-server

(if it says connection refused, it's not running so proceed to the next command):

- adb start-server

(it should say daemon not running; starting now at tcp:(port))

Finally,

- adb devices

If it says no permissions (missing udev rules?), it means your Quest is already being seen by ADB, but your Linux user lacks permission to access it. So, do the following:

- sudo apt update
- sudo apt install android-sdk-platform-tools android-sdk-platform-tools-common
- sudo usermod -aG plugdev $LOGNAME

Now, log out of Mint and log back in.

After logging back in, open the terminal and run command:

- id

It will show if your user is now part of the “plugdev” group with a number in front of it.

Now restart the ADB server:

- adb kill-server
- adb start-server

(it must say daemon started successfully after)

- adb devices

If it still says no permissions, disconnect the Quest 3 from its cable, reboot the PC.

Now issue these two commands:

- adb kill-server
- adb start-server

(it MUST say **Daemon started successfully**!)

Now connect the Quest 3 via cable, and click the “USB Detected” notification (you can safely close the file manager window that pops up). Motion-controller-click Allow (or ALWAYS ALLOW from this computer), and return to the in-headset ALVR app.

Now check it’s working with command on PC:

- adb devices

Now you should see no more warnings and just “List of devices attached” followed by a serial number style string of characters, labelled “device”. That means it’s connected correctly. Quit and relaunch the ALVR desktop app.

In the dashboard, under Wireless Devices, remove the Quest 3's entry to ensure it does not connect wirelessly, and slide Wired Connection to ON. Then click Launch SteamVR from the dashboard.

This will automatically launch steam, steamvr and connect the Quest 3 to it.

Now, finally, desktop viewing is possible as well.

Test out a steamvr native game now, such as Ragnarok, which for me went to a black screen after launch using the wireless connection. Keep graphics settings in steam and alvr the same as before, for now.

For me, Ragnarok still failed to work after showing the intro graphic, as it still went to a black screen. I fixed it and got it working by going to Steam: Ragnaraok: Right-click: Properties: Compatibility: Proton 9.0.

So, lesson for each steamVR native game you try: Try different compatibility options until it works.

SteamVR Games Tested using **Reference 1.2** settings: Beat Saber, Ragnarok (Proton 9.0), Bootstrap Island, Half Life 2: VR Mod, Paradiddle (Proton 9.0 with 'prev_version'), WRATH: Aeon of Ruin VR - Brutal Edition (using default everything).

Reference 1.1: Exact Steps to Successfully connect Quest 3 to SteamVR over cable using ALVR, every time:

1.. Make sure Quest 3 and PC are on the same network, even if PC connects via ethernet.

2.. Open the ALVR app on Quest 3 (WiFi needs to be on for this, even if you'll use the cable).

3.. Start steam independently on Linux Mint.

4.. Launch ALVR on Linux Mint.

5.. Connect the cable from PC to the Quest 3.

6.. in Quest 3, click the "USB Detected" notification.

7.. In ALVR Dashboard on Mint, ensure Wired Connection is ON.

This will launch SteamVR in the headset. Otherwise do it using the ALVR Dashboard on Mint's Launch SteamVR button.

If it crashes, simply return to the ALVR Dashboard on Linux and click Launch SteamVR.

Reference 1.2: Best-quality ALVR+SteamVR settings (Cabled):

1.. On Presets page:

- Resolution preset: Medium
- Refresh rate: 72 Hz
- Encoder preset: Quality
- Foveation preset: Light.
- Codec: H.264

2.. On Video page:

- Passthrough: Off.
- Bitrate mode: Constant
- Bitrate: 1000 Mbps (If any issues with connection, start off with 800Mbps).
- Adapt to framerate: Off.
- History size: 256.
- Image corruption fix: Off.
- Foveated encoding: Leave on. Don't expand details.
- Color correction: Off.
- Maximum buffering: Depends on if there's microstuttering in games. Good default 3, but if there's still microstutters raise it to 4 and test, then try 5. So I'm setting mine to 4 due to micro-stutters in walking in Bootstrap Island.
- Encoder config: :
- -- Leave Quality preset on Quality.
- -- Enable VBAQ/CAQ: Off.
- -- h264: Profile: High
- -- Entropy coding: CAVLC (because CABAC causes pixel-square artefacts).
- -- Filler data: Off
- -- 10-bit encoding: Off (and override off).
- -- Full range color: On (but override off).
- -- Encoding gamma: leave at default of 1 (override off).
- -- Leave the rest as-is.
- Force software decoder: Off
- Transcoding view resolution: Scale: 1.000
- --Note: Want sharper image: 1.1 to 1.2. Getting stutter? Drop to 0.9
- Emulated headset view resolution: Scale: 1.000

- -- Same logic as transcoding above.
- Preferred FPS: Leave at 72Hz
- Clientside foveation: Off
- Client-side post-processing: Off
- Upscaling: Off.

3.. On Connection page:

- Client discovery: ON
- Wired client autolaunch: ON

4.. On Extra page:

- Open and close SteamVR with dashboard: ON (**Do this for Wireless method as well**).

5.. **In SteamVR settings**, when in-headset:

- General: **boost Resolution to 250%** as a good default for all games.

Why? *Because you wouldn't believe the night-and-day difference it makes to how crips the visuals look in most games, such as Bootstrap Island, which went from a blurry mess, to clear and awesome graphics!*

- But, **adjust as necessary** for different games, which you can do on a per-game basis for SteamVR native games, but must adjust this global value for flatscreen-modded-to-VR games.

Et Voila! Now you can install (if not previously installed) and PLAY your VR-native SteamVR games!!

Anecdotal: Reasoning: Why ALVR and not WiVRn?

Answer:

1. Benefits of WiVRn vs ALVR:

Wireless: WiVRn appears to have the better reputation for **latency, frame pacing, and image quality** on Linux in recent user comparisons, but that evidence is mostly anecdotal rather than a controlled benchmark. One recent Linux VR comparison said **WiVRn had lower latency and better picture quality than ALVR** in VRChat, and another GamingOnLinux piece highlighted WiVRn's low-latency transmission and tracking improvements in recent releases.

2. Benefits of ALVR over WiVRn:

Cabled / USB: both support USB, but ALVR's USB path looks **more documented and mature** right now. ALVR has a dedicated official wired-over-USB setup guide built around ADB forwarding, while WiVRn's docs confirm USB support and troubleshooting for USB connections but do not present the same kind of step-by-step "wired mode" setup flow.

3. The problem for Quest 3 + Nvidia GPU (5080):

This is the biggest caveat.

WiVRn's own docs explicitly warn that it is **not properly optimized for NVIDIA GPUs** and that motion latency may be significantly worse above default rendering resolution. That makes it hard to recommend WiVRn as the obvious "higher clarity" winner for Linux Mint + RTX 5080 guide, even if some users report that it looks better than ALVR in practice.

4. Summary of 'Why not use WiVRn' for Quest 3 + Linux + Nvidia (5080):

WiVRn often has the better reputation for wireless image quality and latency, but ALVR's USB/cabled path is more clearly documented and is the safer recommendation, **especially on NVIDIA systems**, because WiVRn on Nvidia GPUs has major motion latency and rendering resolution issues.

5. **What it means for this guide**:

It means, at least for version 1 of this guide, I shall not include instructions on how to replace ALVR with WiVRn, or have both. However, I will include a full step-by-step guide in the next version of this document.

Anecdotal 2: How to clean up unwated .desktop files from Cinnamon menu

For whenever you make and edit **.desktop** files, if at any time already-deleted .desktop files appear in search in the Cinnamon menu, and you've definitely deleted them everywhere on the filesystem, **you can erase these entries by** issuing these commands, then changing a Settings option:

- rm ~/.local/share/recently-used.xbel
- touch ~/.local/share/recently-used.xbel
- rm -rf ~/.cache/menus ~/.cache/cinnamon
- cinnamon --replace &

Now open the **System Settings** app, go to Privacy, and switch off 'Remember recently accessed files' if you, y'know, **KNOW already HOW TO FIND YOUR BLOODY FILES ON YOUR OWN**.

---

14. How to Add and Pin a Window in-game using Quest 3 and ALVR:

By default for other headsets and using **MS Windows**' SteamVR, there's always a "+" button on the bottom panel of the in-headset SteamVR interface, but it's completely missing here.

This appears to be a known issue/bug/limitation with SteamVR on Linux, currently.

A reportedly better and more stable way to add and pin a window, therefore, is to use a tool called WayVR. However, it has some major limitations because it really requires wiVRn, an alternative solution to ALVR, for Monado's OpenXR runtime that WayVR relies on, to work properly.

How to get WayVR working with ALVR:

a) I**nstall WayVR**:

First, go to https://github.com/wayvr-org/wayvr

and click the Releases link to download the latest version of WayVR (currently v26.2.1 for me) in the form of the AppImage (so e.g. WayVR-v26.2.1-x86_64.AppImage).

Download it to ~/Applications/

Second, get it working for SteamVR:

Note: I would typically convert the AppImage file into a real app for ease of access and pin-ability to the panel or dock, if using one, but in this case, WayVR does not open as a normal app, even if converted to one, so it's pointless to do so, as was done for Satisfactory Mod Manager in the UEVR section.

1.. Give it execute permissions:

- chmod +x ~/Applications/WayVR*.AppImage

Note: You can check the read/write/execute permissions of a file at any time using command "ls -l" followed by the folder path, so in this case:

- ls -l ~/Applications/

There you should now see WayVR's AppImage file with permissions -rwxr-xr-x, meaning it's executable (x) by 'owner' (you), 'owner's group', and 'others', from left to right. You can see the 'owner' and 'group owner' listed to the right of those permissions where it should say your username twice.

Now, double click the file in the file manager to start it running.

Third, to test that it's working, get into SteamVR in the headset (see 'Reference 1.1: Exact Steps to Successfully connect Quest 3 to SteamVR over cable using ALVR, every time' above).

Once you have that ready, with SteamVR open in headset, on desktop, in the file manager, navigate to ~/Applications and double-click the WayVR AppImage file.

You can then go to SteamVR settings: Startup / Shutdown and there in Choose Startup Overlay Apps, you should see WayVR which is set to start up automatically with SteamVR's launch, by default. If that doesn't appear, exit SteamVR, and relaunch it.

b) **Use WayVR**:

To open the interface in-game, double tap the Y button on your left Quest 3 controller.

c) **How to open a browser window**:

Open WayVR in-game, by double-tapping Y. The keyboard will appear and you can twist your left wrist to access the controls. Add a new 'set' using the keyboard hamburger menu, then use your wrist to open the bottom-left icon's link to Monado, basically. There navigate to apps and choose

your browser of choice. I tested mine with Ungoogled Chromium, but feel free to test with any browser. Once selected, that browser will come up.

You can then log in to Twitch or YouTube, for example, to have some accompaniment while gaming in VR! You can float the window anywhere you like!

If you are a streamer, log in to Twitch, go to your chat, click the cog; **pop-out a chat window**, and then you have two options:

a) grab-button-move the window to up **above your normal field of view to float in the world above where you normally look**.

or

b) **grab and move it with your right controller somewhere past your left controller's wrist**, then click its edit button next to its top-right 'close' button, and select the balloon icon (Positioning button) and choose '**Follow the left hand**'. Then you can **twist your left wrist right** a bit, use the grab action on your right controller to **position the window to the left of your left hand**, and then untwist your left wrist so that the window is **quite hidden during normal gameplay and articulation**.

Limitations:

WayVR using our setup is extremely limited in what functions and what does not. Thus, you may notice the option to, and want to, get a thin and tall window in view so it aligns with the more typical shape of a Twitch chat window. This is NOT possible without Wayland (which is broken on Linux Mint) or WiVRn, which is a whole replacement alternative to ALVR and has reported issues with Nvidia GPUs.

As mentioned, I will not include a tutorial on getting WiVRn installed, either to replace ALVR or alongside it, in this first version of my guide.

But, I will probably include this massive extra step (that involves starting from scratch with WiVRn, as an alternative/alongside ALVR, that would make WayVR function as designed) in the next version of this guide, for completeness, and because I also want to see for myself if it works for Mint + SteamVR + Nvidia 5080, and how it compares to ALVR.

Part 2: UEVR Guide for Linux VR

Essential: Install protontricks:

- sudo apt install protontricks

First, let's test UEVR on a game that not only requires it, but an extra mod as well: **Satisfactory**.

Important Note: **Part 2 is meant to be read in order**, so that you obtain a firm understanding of dealing with UEVR-based mods (all three games make use of a form of UEVR), **and don't miss any important steps**, even if you don't plan to play any particular game.

UEVR Game Tutorial 1: Satisfactory (with Dortamur's mod)

Step 0: Install Satisfactory on Steam!

Launch the game once in flatscreen to ensure all necessary Steam game/config files/folders are present, that we make use of below.

Note: Save all these files to ~/VRonMint/SatisfactoryVR/ (create this subfolder)

First, download (& then configure) the Satisfactory Mod Manager linux app:

https://ficsit.app/

Make it executalbe:

- chmod +x ~/VRonMint/SatisfactoryVR/SatisfactoryModManager_linux_amd64.AppImage

Now launch it:

- ~/VRonMint/SatisfactoryVR/SatisfactoryModManager_linux_amd64.AppImage

Make sure it works, and close the app without changing anything.

Second, let's make this 'AppImage' file into a real system-wide app able to stick in the panel/dock and launch from Cinnamon menu: (This applies to all AppImage files you come across):

1. Make an Applications folder in Home directory, using the file manager or:

- mkdir -p ~/Applications

2. Move the AppImage file to the applications folder, using the file manager, or:

- mv ~/VRonMint/Satisfactory/VR/SatisfactoryModManager_linux_amd64.AppImage ~/Applications

3. Create a .desktop file for it:

- xed ~/.local/share/applications/SatisfactoryModManager.desktop

Paste in the following:

```
[Desktop Entry]
Version=1.0
Type=Application
Name=Satisfactory Mod Manager
Comment=Created from official ficsit AppImage
Exec=/home/(YourUsername)/Applications/
SatisfactoryModManager_linux_amd64.AppImage
Terminal=false
Categories=Game;
StartupNotify=true
```

Save and close the text editor.

4. Make the .desktop file executable:

- chmod +x ~/.local/share/applications/SatisfactoryModManager.desktop

5. Let's get the icon for the app from the AppImage: (Note: **If you don't care about the app having an icon, skip to and do Steps 7 and 8**).

a) Copy and paste the AppImage file to a new folder on Desktop called SatisfactoryModManagerIconExtraction

b) Issue this command to get the guts:

- cd ~/Desktop/SatisfactoryModManagerIconExtraction/
- ./Satisfactory*.AppImage --appimage-extract

This creates a folder called squashfs-root

c) Copy the icon image in that folder to the default icons storage folder:

- cp squashfs-root/SatisfactoryModManager.png ~/.local/share/icons/SatisfactoryModManager.png

6. Lastly, link to the icon in the .desktop file:

- xed ~/.local/share/applications/SatisfactoryModManager.desktop

Add this line below the "Exec=..." line:

- Icon=/home/**(YourUsername)**/.local/share/icons/SatisfactoryModManager.png

7. Update the apps database:

- update-desktop-database ~/.local/share/applications

8. Restart Cinnamon (If the desktop doesn't come back after, just restart the computer):

- cinnamon --replace &

NOW, it will appear in the Games category of the Cinnamon Menu and in search with an icon.

Click it to test that it opens up!

Now, while open, you can right-click it in the panel/dock and pin it (or "keep in dock" if using Plank Reloaded).

Choose Wait for me to press "Apply", for better control. Then click the Get Started Button.

Third, get and install Dortamur's UEVR Enhancements mod:

Either go to this page:

https://ficsit.app/mod/UEVREnhancements

and click the Install button to the right under "Latest Versions".

OR, within the Satisfactory Mod Manager, the UEVR Enhancements by Dortamur mod is the second one on the list for me, so there you can click the white download button.

Either way, then click **Apply one change**, until downloaded and a green "Play Satisfactory" appears in the place of the apply button.

Fourth, get UEVR nightly from:

https://github.com/praydog/UEVR-nightly/releases

I'm using the latest, which is currently UEVR Nightly 01127. Click and download uevr.zip

Rename it to uevrNightly01127.zip

Extract its contents to a new subfolder by the same name, using the file manager, or:

- unzip ~/VRonMint/uevrNightly01127.zip -d ~/VRonMint/uevrNightly01127/

Fifth, download the Satisfactory profile for UEVR for Steam zip file, to the SatisfactoryVR folder:

https://github.com/dortamur/satisfactory-uevr-enhancements/raw/master/UEVR/FactoryGameSteam-Win64-Shipping.zip

You can see this link on Dortamur's Getting Started page on Github:

https://github.com/dortamur/satisfactory-uevr-enhancements/wiki/Getting-Started

(If the direct link from this document doesn't work).

Sixth, download this Microsoft dotnet .exe file, to the same SatisfactoryVR folder:

https://builds.dotnet.microsoft.com/dotnet/WindowsDesktop/6.0.25/windowsdesktop-runtime-6.0.25-win-x64.exe

Seventh, get the known-compatible Proton-GE10-17 release from:

https://github.com/GloriousEggroll/proton-ge-custom/releases/tag/GE-Proton10-17

Extract the 'GE-Proton10-17.tar.gz' file using the Terminal to the already-existent steam compatibility tools folder, using command:

- tar -xf ~/VRonMint/SatisfactoryVR/GE-Proton*.tar.gz -C ~/.steam/steam/compatibilitytools.d/

(Remember the -C option between the two locations).

Exit Steam.

Launch steam again, after at least 30 seconds.

Navigate to Satisfactory: Right-click: Properties: Compatibility: Click Force use of a specific Steam Play compatibility tool, click the drop-down list, and scroll all the way to the bottom and click GE-Proton10-17. You'll see the game getting updated via steam download. Now close the properties box.

Eighth, install the Microsoft dotnet runtime .exe using this long-ass Terminal command (copy and paste the whole thing as one:

```
SteamAppId=526870 SteamGameId=526870 \
STEAM_COMPAT_CLIENT_INSTALL_PATH="$HOME/.steam/debian-installation" \
STEAM_COMPAT_DATA_PATH="$HOME/.steam/debian-installation/steamapps/compatdata/
526870" \
"$HOME/.steam/steam/compatibilitytools.d/GE-Proton10-17/proton" run \
"$HOME/VRonMint/SatisfactoryVR/windowsdesktop-runtime-6.0.25-win-x64.exe"
```

It may throw some warnings, but it's been successful if the Microsoft installer window comes up. Click install, let it install, then click the Close button when installed.

Ninth, time to test out the game, launch UEVR, configure and inject it!

|**See Reference 1.1** above to get connected to in-headset SteamVR.

Launch Satisfactory from Quest 3's SteamVR.

Let it compile shaders, or skip for now.

In game, go to **Options: Video: set it to DirectX 11**.

Set the graphics preset to High to begin with.

Fully exit the game.

Launch the game again.

Load a save game or start a new game and wait until you are in the world.

It will say "Ready for UEVR injection" in the top right.

Now, alt-tab to the Terminal on Linux and issue this command as one copy and paste:

```
SteamAppId=526870 SteamGameId=526870 \
STEAM_COMPAT_CLIENT_INSTALL_PATH="$HOME/.steam/debian-installation" \
STEAM_COMPAT_DATA_PATH="$HOME/.steam/debian-installation/steamapps/compatdata/
526870" \
"$HOME/.steam/steam/compatibilitytools.d/GE-Proton10-17/proton" run \
"$HOME/VRonMint/SatisfactoryVR/uevrNightly01127/UEVRInjector.exe"
```

This will open the Injector (UEVR) app.

Click “**Import config**”: Click the / folder, then home -> your username -> VRonMint -> Satisfactory/ and select the **FactoryGameSteam-Win64-Shipping.zip** file.

Select OpenXR as the runtime in the UEVR box.

Make sure **Nullify VR plugins is OFF**.

Scroll down to about 2/3rds of the way down, and **checkmark t**he box labelled **VR_GhostingFix**. This helps eliminate shakiness in the world, that I found to be present without it.

Now click Inject.

After a few seconds, Satisfactory will open in the background. Use SteamVR’s bottom panel to switch to it.

Also, if you haven’t previously, ensure resolution in steamVR is set to 150-250%, and Satisfactory in-game graphics are set to High or Ultra. But feel free to play around with these settings to get the best possible graphics and smoothness solution.

Enjoy!

Reference 2.1: Satisfactory startup steps, each time:

1.. Make sure Quest 3 and PC are on the same network, even if PC connects via ethernet.

2.. Open the ALVR app on Quest 3 (WiFi needs to be on for this, even when using cable).

3.. Start steam independently on Linux Mint.

4.. Launch ALVR on PC.

5.. Connect the cable from PC to the Quest 3.

6.. In ALVR Dashboard on Mint, slide the slider for Wired Connection to ON, if not on already.

7.. in Quest 3, click the “USB Detected” notification. This should now launch SteamVR in the headset. Otherwise do it using the ALVR Dashboard on Mint’s Launch SteamVR button.

If it crashes, simply return to the ALVR Dashboard on Linux and click Launch SteamVR.

8.. From SteamVR in headset, launch Satisfactory.

This opens your monitor’s desktop view which is shortly covered by a giant Satisfactory sign, and then by a window of Satisfactory starting up flat.

9.. Using the mouse, open your save file or create a new one and load into the world.

10.. Pause using escape key, and alt-tab to the Terminal, and paste in this long command in one go, to launch the UEVR app:

```
 SteamAppId=526870 SteamGameId=526870 \
STEAM_COMPAT_CLIENT_INSTALL_PATH="$HOME/.steam/debian-installation" \
STEAM_COMPAT_DATA_PATH="$HOME/.steam/debian-installation/steamapps/compatdata/
526870" \
"$HOME/.steam/steam/compatibilitytools.d/GE-Proton10-17/proton" run \
"$HOME/VRonMint/SatisfactoryVR/uevrNightly01127/UEVRInjector.exe"
```

11.. UEVR settings should be the same as last time, but feel free to review them.

12.. Click Inject. Wait a few seconds, and Satisfactory will appear in 3D behind SteamVR’s in-headset interface. Use the SteamVR bottom panel to click the Satisfactory app icon using your motion controller.

Et Voila!

UEVR Game Tutorial 2: GTA San Andreas - Definitive Edition

Represents: *A Typical UEVR Game that only needs UEVR injection, no additional mods.*

Step 0: Install the game!

Launch and run the game one time in flatscreen. Make sure it's working!

Now for the VR steps:

Right-click GTA San Andreas in your Steam Library on desktop, and click Properties: Updates. There, you'll find the App ID which I've included in the long ass command for SteamAppId=, SteamGameId= and the line that starts with STEAM_COMPAT_DATA_PATH=, to launch UEVR below. Do this for each UEVR game you want to play. The rest of the command is the same each time.

This assumes your default compatibility tool in Steam: Settings: Compatibility is set to Proton Experimental, and that you don't add anything nor change any settings in GTA's properties.

Next, adapt the command from Satisfactory's tutorial "Eighth, install the Microsoft dotnet runtime" part, to install the Microsoft .NET 6.x desktop runtime inside the GTA SA - DE proton prefix compat data folder.

But first open your file manager and navigate to ~/VRonMint/Satisfactory/ and copy windowsdesktop-runtime-6.0.25-win-x64.exe to ~/VRonMint/ for acccess to all UEVR games just to ensure independence and disassociation from Satisfactory and its files and folders.

Second, do the same for uevrNightly01127.zip - copy it to ~/VRonMint/ and then extract it into its own folder there with command:

- unzip ~/VRonMint/uevrNightly01127.zip -d ~/VRonMint/uevrNightly01127/

Now install .Net 6.x into GTA SA's compat folder. A necessary step for each UEVR game, BECAUSE UevrInjector.exe is a .NET app, so it needs it, to work on Linux:

```
SteamAppId=1547000 SteamGameId=1547000 \
STEAM_COMPAT_CLIENT_INSTALL_PATH="$HOME/.steam/debian-installation" \
STEAM_COMPAT_DATA_PATH="$HOME/.steam/debian-installation/steamapps/compatdata/
1547000" \
"$HOME/.steam/debian-installation/steamapps/common/Proton - Experimental/proton"
run \
"$HOME/VRonMint/windowsdesktop-runtime-6.0.25-win-x64.exe"
```

The Microsoft Windows Desktop Runtime - 6.0.25 (x64) Installer window will come up. Click Install. Once it's done, click Close.

Now, use **Reference 1.1** to get into SteamVR in your Quest 3.

Launch GTA San Andreas Definitive Edition.

On very first launch, you'll have to look at the desktop to complete setup (accept language etc). It will probably crash. That's fine just launch it again.

Now, alt-tab to the Terminal on Linux and issue this command as one copy and paste:

```
SteamAppId=1547000 SteamGameId=1547000 \
STEAM_COMPAT_CLIENT_INSTALL_PATH="$HOME/.steam/debian-installation" \
STEAM_COMPAT_DATA_PATH="$HOME/.steam/debian-installation/steamapps/compatdata/
1547000" \
"$HOME/.steam/debian-installation/steamapps/common/Proton - Experimental/proton"
run \
"$HOME/VRonMint/uevrNightly01127/UEVRInjector.exe"
```

Select OpenXR as the runtime in the UEVR box.

Make sure **Nullify VR plugins is OFF**.

Scroll down to about 2/3rds of the way down, and **checkmark t**he box labelled **VR_GhostingFix**.

Now click Inject.

After a few seconds, GTA San Andreas Definitive Edition will open in the background. Use SteamVR's bottom panel to switch to it.

I found that setting SteamVR's resolution to 300% worked well for GTA SA DE, and in-game graphics are set to High. However, with default graphics settings, everything is a little bit shiny in a weird way. I turned a bunch of the settings towards the bottom of the game's graphics options OFF, and that made the weird reflectiveness/shininess go away. I also turned Clouds down to Volumetric instead of the highest setting. But everything else I changed was an ON or OFF option. Try it out and find what gives you the best visual experience.

A note on trying different proton versions for best fit experimentation:

You could also try modifying which version of Proton is used in the command to launch UEVRInjector.exe

One source to get an idea of which proton versions others have successfully used to get the game running, at least flatscreen on their (different distros of) Linux systems is https://www.protondb.com/

..where you can search for the game to get an idea of the consensus of the most commonly used proton/proton-GE version used.

The reliable way to ensure things work as expected when switch proton versions for a game is (for example with Proton 9.0):

1. In Steam, set **GTA San Andreas Definitive Edition** to **Proton 9.0** in Properties > Compatibility.

2. Reinsatll the Microsoft .Net 6.x runtime using the proton version of your choice.

In this case, the command would be:

```
SteamAppId=1547000 SteamGameId=1547000 \
STEAM_COMPAT_CLIENT_INSTALL_PATH="$HOME/.steam/debian-installation" \
STEAM_COMPAT_DATA_PATH="$HOME/.steam/debian-installation/steamapps/compatdata/
1547000" \
"$HOME/.steam/debian-installation/steamapps/common/Proton 9.0 (Beta)/proton" run
\
"$HOME/VRonMint/windowsdesktop-runtime-6.0.25-win-x64.exe"
```

---Adapt the command to fit your chosen proton version.

3. In your injector command, also change the Proton path to **Proton 9.0**.

--To find the path to Proton 9.0's installation folder, in Steam Library, ensure "Tools" is checkmarked, then right-click Proton 9.0: Manage: Browse Local Files.

In that folder will be the main executable for Proton 9.0 called simply "proton".

For me, for some reason my Proton 9.0 (Beta) installation folder was installed on my second nvme drive, at /mnt/1TBnvme/SteamLibrary/steamapps/common/Proton 9.0 (Beta)/

Therefore, my new UEVRInjector.exe launch command that I Alt-tab and enter in the terminal when ready to inject a game into UEVR, for GTA SA DE, would be:

```
SteamAppId=1547000 SteamGameId=1547000 \
STEAM_COMPAT_CLIENT_INSTALL_PATH="$HOME/.steam/debian-installation" \
STEAM_COMPAT_DATA_PATH="$HOME/.steam/debian-installation/steamapps/
compatdata/1547000" \
"/mnt/1TBnvme/SteamLibrary/steamapps/common/Proton 9.0 (Beta)/proton" run \
"$HOME/VRonMint/uevrNightly01127/UEVRInjector.exe"
```

If GTA were to be launched by Steam with Proton 9.0, but you inject with Experimental, you are mixing runtimes. Sometimes that still works, but it is **not the clean setup** and can absolutely be a source of weird detection/injection issues. Matching the injector Proton to the game Proton is the safer approach. A similar principle applies to the Microsoft .NET 6.x runtime - it might work just fine as-is, but it's the **path-of-least-error** to do step 2 above, for each new version of proton you want to try, to avoid any potential issues.

Proton GE Versions:

If you want to make use of a GE-Proton version, to try and see if one of those would be any better, you can download any version at:

https://github.com/GloriousEggroll/proton-ge-custom/releases

Download the 'GE-Proton(VersionNumber).tar.gz' file to your ~/VRonMint/ folder, and extract it using the Terminal to the already-existent steam compatibility tools folder, using command:

- tar -xf ~/VRonMint/GE-Proton(VersionNumber).tar.gz -C ~/.steam/steam/compatibilitytools.d/

(Remember the -C option between the two locations).

Exit Steam.

Launch steam again, after at least 30 seconds.

First, Navigate to the game you'd like to try this GE-Proton version on: Right-click: Properties: Compatibility: Click Force use of a specific Steam Play compatibility tool, click the drop-down list, and scroll all the way to the bottom and click GE-Proton(VersionNumber). You'll see the game getting updated via steam download. Now close the properties box.

Second, if it's a GE-Proton version you have not previously installed Microsoft .NET 6.x into for that particular game (as each game has its own compat folder), then do so now, using step "2." above, below the title of "A note on trying different proton versions for best fit experimentation".

Third, you would modify your UEVRInjector.exe launch command to include the new GE-Proton version, **so for example**, having used the extraction command above, **GE-Proton10-34** (the latest at time of writing), would be installed at ~/.steam/steam/compatbilitytools.d/GE_Proton10-34/, so AFTER changing proton version in Steam Library for GTA SA DE Properties, and installing .NET 6.x runtime, you would launch the game, then alt-tab and issue the following command block, as one:

```
SteamAppId=1547000 SteamGameId=1547000 \
```

```
STEAM_COMPAT_CLIENT_INSTALL_PATH="$HOME/.steam/debian-installation" \
STEAM_COMPAT_DATA_PATH="$HOME/.steam/debian-installation/steamapps/
compatdata/1547000" \
"$HOME/steam/steam/compatbilitytools.d/GE_Proton10-34/proton" run \
"$HOME/VRonMint/SatisfactoryVR/uevrNightly01127/UEVRInjector.exe"
```

That should be more than sufficient to get the principle of using UEVR on Linux Mint with ALVR and a 5080 GPU or similar Nvidia card. But, as a bonus, here’s a step-by-step guide to getting Resident Evil 2 Remake working on Linux Mint 22.3 + ALVR + Nvidia 5080...

Next UEVR Game Tutorial: Resident Evil 2 (Remake)

Note: I tried RE2 the standard UEVR way, just as done with GTA San Andreas DE above, but UEVR failed to “detect a uevr instance”. Thus, that path was abandoned in favour of Praydog’s REFramework “mod”.

I tried the ray-tracing standard version, but while the game did load into 3D VR, the REFramework menu button would not open any menu, nor the controller menu activation method, and Leon was transparent like a ghost.

Thus, I’ve ditched those previous instructions, which serve no purpose here (as I’ve had to write and rewrite various parts of this guide before finding what works), so these are the WORKING instructions:

1. Install Resident Evil 2 (remake) on Steam.

Then, right-click it in Steam Library: Properties: Game Versions & Betas: Click “dx11_non-rt” and Steam will automatically update the game, pretty much a full re-download.

2. Run the game once in flat-screen mode, and get to a playable part of the game. Quit the game.

3. install REFramework VR’s latest nightly release from:

https://github.com/praydog/REFramework-nightly/releases

For me it’s REF Nightly 01320, which is what this tutorial will be based on.

Download, specfically the file “RE2_TDB66.zip” **with** the TDB suffix, which are the versions **without** ray-tracing, to ~/Downloads. Then:

- cd ~/Downloads
- unzip -o RE2_TDB66.zip -d “(path-to-steam’s-RE2-install-Folder)”

You can get this path by right-clicking RE2 in the steam library and click Manage:Browse Local files. So for me the command is:

- unzip -o RE2_TDB66.zip -d /mnt/1TBnvme/SteamLibrary/steamapps/common/RESIDENT\ EVIL\ 2\ \ BIOHAZARD\ RE2/

..each \ represents a space.

4. Set the required Steam launch option:

REFramework's GitHub page instructions for Proton/Linux are to add this exact launch option to Steam after the above unzipping. Do so in Steam Library: RE2: Right-click: Properties: General.

```
WINEDLLOVERRIDES="dinput8.dll=n,b" %command%
```

5. Use Reference 1.1 to run the game.

If the mod loads as it should, you'll see RE2 in 3D open behind the steam interface. Switch to it using the bottom bar of the steam interface.

Note: The **RE2 in-game PAUSE MENU** is the SAME BUTTON as the SteamVR button on your left Quest 3 controller. Just tap it, **choose Return to Game**, and you'll be in the RE2 in-game pause menu.

Now, you should be able to open the REFramework menu by pointing at your left hand palm with your head AND your right controller.

If that doesn't work, hit the Insert key on your physical keyboard.

Well, neither worked for me, but at least using the non-ray-tracing version of praydog's RE2 mod plus the 'dx11_non-rt' Steam game version means controller support works, although the button layout is different to the prompts on-screen, which is why I prefer to play with a gamepad, AND Leon does not appear as a transparent ghost, as he did when I tried the ray tracing version and mod.

6. To get the best graphics while maintaining stability, I set SteamVR resolution to 250% and chose DX11 in the main menu, in-game. Play around with the different in-game graphic settings to see what works, and optionally try different steamVR resolutions in combination, as well.

7. Now let's fix the REFramework not showing up in-headset:

Use the usual startup loop to get SteamVR running on your Quest 3.

Launch RE2. Once running, press the SteamVR button on your left Quest 3 controller. Select Controller Bindings: Custom: Choose another: Oculus Touch (even if it was already on that), and then I selected Show Binding on Controllers, and only THEN was I able to see the REFramework menu by pointing my right controller at my left palm. I hope that works for you too!

Now you can close the bindings displayer, load into a game, and open this menu at any time to adjust settings! I'll leave it to you to find out the best settings to use in the REFramework menu.

So, that shall suffice for the purpose of this tutorial.

Conclusion

In part 1, you saw how to get set up with Linux Mint, set up Steam and SteamVR, followed by ALVR and how to play SteamVR-native games on your Quest 3 Wirelessly and Wired.

After completing part 2, you now have the tools to play any of the modern Resident Evil game supported by PrayDog's REFramework mods, at:

https://github.com/praydog/REFramework

Besides that, **you've learned how** to play **Satisfactory** with its custom Dortamur mod on top of the typical UEVR install, and then you've also seen **how to play a "vanilla" UEVR game** that only requires UEVR injection to function (in the example of GTA San Andreas DE).

All on Linux.

Without Windows.

Ever again.

If preferred.

At the cost of a bit of graphical fidelity.

But at the gain of your freedom from Microsoft.

If you've found this guide helpful, come find me on one of the acknowledgement Twitch streams sometime, and say hi. Perhaps I'll stream myself sometime, so that I can include a How-To on streaming to Twitch from Linux Mint in either another Guide document, or a future version of this one.

-- *Future on VR*